Relational Operators

# Broker

## Broker [object]

Use the object *Broker* for brokering offered services and required services. You might model the **manager** of a **plant**, the **supervisor** of a **department**, or the **foreman** of a **shop** with it.

### Image

### Remarks

The *Broker* cooperates with the **Exporter** and the *importers* (see **Tab Importer** and **Sub-tab Failure**) of the *Station*, the *ParallelStation*, the *AssemblyStation*, the *DismantleStation*, the *DePortioner*, the *Mixer*, the *Portioner*, and the *Tank*.

The *Broker* is the go-between for services offered and services required. You might model the manager of a **plant**, the **supervisor** of a department, or the **foreman** of a shop with it. Each *Broker* can manage several *Exporters*/*Workers*, which tender services, and it may receive requests from several *importers* that require services. A request consists of a list of the required services and the amount of required services. A service name is a *string*.

When a *Broker* receives a request from an *importer*, it immediately attempts to fulfill it using the *Exporters*/*Workers* it manages. If the *Broker* does not succeed in doing this, it may also pass the request on to other *Brokers* with which it is connected with a *Connector*. The direction of the *Connector* determines the direction in which *Plant Simulation* passes the request on. If the *Exporters*/*Workers* of different *Brokers* are able to satisfy the request, those *Exporters*/*Workers* are assigned. If the request cannot be satisfied immediately, the *Broker* that received it first will save it.

The *Broker* then attempts to provide the service at a later point in time. To do this, he goes from requested service to requested service and attempts to find an *Exporter*/*Worker* for each. If an *Exporter*/*Worker* offers the service, it is going to reserve the maximum possible number or the necessary number. Then the *Exporter*/*Worker* cannot offer this reserved amount of services for other services. For this reason we recommend to request special services first and more general ones last.

> **Note**
>
> If the *importer* requests several services at the same time, all of these services have to be available at the same time, before the *Broker* assigns them to the station.

The *Broker* does not run any optimization! That way it may happen that the *Broker* cannot assign any *Exporters*/*Workers*, although the *Exporters*/*Workers* may theoretically be assigned.

Compare the following example:

The *importer* named *MyImporter* requires one service A and B each. There are two *Exporters/Workers* with a capacity of 1 each. *Exporter1* exports services A and B, *Exporter2* only exports service A. Both *Exporters/Workers* are registered with the same *Broker*. If the *importer* named *MyImporter* requests the services in the order (A,B), then the *Broker* assigns *Exporter1* for service A and realizes that *Exporter2* cannot export the still missing service; thus it cannot fulfill the request.

If the *importer* named *MyImporter* puts in the request in the order (B,A), it will be fulfilled immediately: *Exporter1* provides service B and *Exporter2* provides service A. The sequence in which a *Broker* passes requests on is defined uniquely by the *Connectors*. The *Broker* first passes the request on to all of its direct successors, defined by the number of the *Connector*. After that it passes the request on to the successors of the successors. When an *Exporter/Worker* registers with its *Broker* as being available, the *Broker* attempts to find new *importers* for the *Exporter/Worker*. First, the *Broker* checks the unsatisfied requests it manages, and then it has its *Broker* successors check for any unsatisfied requests.

You can also show the **Broker Statistics Table** of a *Broker* in an **HtmlReport**, compare **Display a Broker**.

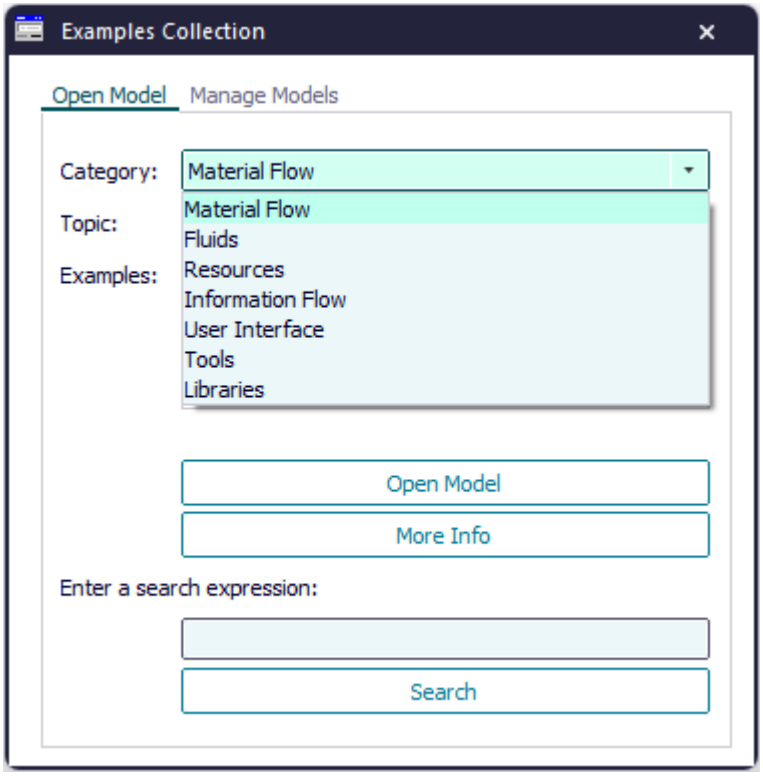
To show a **tooltip** with information about the *Broker*, hover with the mouse over it.

To change the length of the graphic and the anchor points of the *Broker*, click **Show Manipulators**  on the **Edit** ribbon tab or press **M** on the keyboard.

### Add the Object to the Simulation Model

To add the object *Broker* to your simulation model, click  **Manage Class Library** > **Basic Objects > Resources > Broker** on the **Home** ribbon tab.

Compare the sample models: Click the **Window** ribbon tab, click **Start Page > Getting Started > Example Models > Small Examples**. Then, select the respective **Category**, the **Topic**, and the **Example** in the dialog **Examples Collection**, and click **Open Model**.




**See also**

**How the Broker Mediates Jobs to the Worker**

**How the Worker Decides Where to Work**

**Model Workers and the Jobs They Do**

**Model Workers with Importer, Broker, and Exporter**

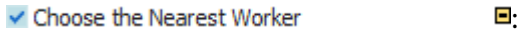
## How the Broker Mediates Jobs to the Worker

Describes how the *Broker* mediates jobs to the *Worker*.

### Mediation Criteria

The *Broker* mediates jobs to the *Worker* according to the **Priority** of the work order.

During mediation, the *Broker* runs through the following phases. In doing so, he differentiates if **Choose the Nearest Worker [check box]** is selected or not selected .

- **Choose the Nearest Worker** is activated  :
  1. **Initial Phase**: The *Worker* is located at the correct station on any *Workplace* suited for the required service. The station is located within the *Worker's* **Scope**.
  2. **Phase 1**: The *Worker* is located at the correct station, but on a *Workplace* that does not support the required service. The station is located within the *Worker's* **Scope**.
  3. **Phase 2**: The *Worker* is not located on a *Workplace*, the station is located within the *Worker's* **Scope** though.
  4. **Phase 3**: The *Worker* is located at another station on a *Workplace*, and the correct station is located within the *Worker's* **Scope**.
  5. **Phase 4**: The *Worker* is located at the correct station, the *Workplace* is suitable for the required service, and the *Worker* does not have a defined **Scope**.
  6. **Phase 5**: The *Worker* is located at the correct station, but on a *Workplace*, which does not support the required service. In addition the *Worker* does not have a defined **Scope**.
  7. **Phase 6**: The *Worker* does not have a **Scope** and is not located on a *Workplace*.
  8. **Phase 7**: The *Worker* does not have a **Scope** and is located on a *Workplace* at the wrong station.
  9. **Phase 8**: The *Worker* is working at the moment and has to be drawn off from his current job.
- ◦ **Choose the Nearest Worker** is not activated :
  1. **Initial Phase**: The *Worker* is located at the correct station on any *Workplace* suited for the required service. The station is located within the *Worker's* **Scope**.
  2. **Phase 1**: The *Worker* is located at the correct station, but on a *Workplace* that does not support the required service. The station is located within the *Worker's* **Scope**.
  3. **Phase 2**: The *Worker* is not located on a *Workplace*, the station is located within the *Worker's* **Scope** though.
  4. **Phase 3**: The *Worker* is located at another station on a *Workplace*, and the correct station is located within the *Worker's* **Scope**.
  5. **Phase 4**: The *Worker* is located at the correct station, the *Workplace* is suitable for the required service, and the *Worker* does not have a defined **Scope**.
  6. **Phase 5**: The *Worker* is located at the correct station, but on a *Workplace*, which does not support the required service. In addition the *Worker* does not have a defined **Scope**.

7. **Phase 6**: The *Worker* does not have a **Scope** and is not located on a *Workplace*.
8. **Phase 7**: The *Worker* does not have a **Scope** and is located on a *Workplace* at the wrong station.
9. **Phase 8**: The *Worker* is working at the moment and has to be drawn off from his current job.

#### See also

**How the Worker Decides Where to Work**

**Worker [object]**

**Workplace**

**Priority [Worker]**

**Priority [SimTalk] - Worker**

**Tab Scope**

**Scope [SimTalk] - Worker**

## Dialog Box of the Broker

### Dialog Box of the Broker

Double-click the icon of the *Broker* to open its dialog box.

#### Edit Simulation Properties

In the dialog box you can change the **simulation properties** of the object. The shared properties are described under **Dialog Items of the Objects**.

#### Edit Animation Properties

To edit the **3D properties** of the object in the dialog box **Edit 3D Properties**:

- Click the button  **Edit 3D Properties** in the lower left corner of the simulation properties dialog box.
- Select the object in the model and press the **spacebar**.

To manipulate the graphic of the object, click **Show Manipulators** on the **Edit** ribbon tab or press **M** on the keyboard.

## Tab Attributes

### Tab Attributes [Broker]

The tab **Attributes** provides the settings, which the object offers.

The tab **Attributes** provides these settings:

**Choose the Nearest Worker [check box]**

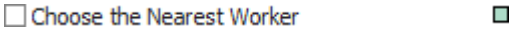
**Importer Request Control [Broker]**

**Exporter Request Control [Broker]**

The shared properties are described under the **Tab Attributes**.

## Choose the Nearest Worker [check box]

To make the *Broker* prefer the *Worker*, who has to walk the **shortest path** to the *Workplace*, select this check box. Clear the check box to make the *Broker* select any *Worker* who can do this job, no matter where he is located at the moment.




### Remarks

When the *Broker* receives an **import request**, it selects *Workers*, who can fulfill the requested services according to the following criteria:

- Highest **Priority**
- Already located at a **Workplace** of the requesting station
- Matching **Scope**

If you select the check box **Choose the Nearest Worker** the *Broker* uses an **additional criterion** for selecting *Workers*: Which of the *Workers* has a to walk a **shorter distance** to the *Workplace* than any of the other *Workers*.

*Plant Simulation* computes routes to the suitable *Workplaces* of the station for all eligible *Workers* while mediating them. For this reason, enabling **Choose the Nearest Worker** can slow down the simulation.

### SimTalk

**ChooseNearestWorker [SimTalk]**

**Scope [SimTalk] - Worker**

### See also

**How the Broker Mediates Jobs to the Worker**

**How the Worker Decides Where to Work**

**Tab Scope**

**Priority [Worker]**

**Objects [Scope]**

**Workers to Create** table

#### Video on YouTube

https://youtu.be/gU1pqu7sWA8?si=wA6ISMbfc9ZVvLme&t=204

## Controls of the Broker

### Controls [Broker]

Click the ellipsis button and select a *Method* in the dialog **Select Object**. Type in the source code of the respective control.

#### Select the Path to an Existing Method

Click the ellipsis button. Navigate to the location of the *Method* in the dialog **Select Object [for controls]** and click **OK**. This inserts the name of the *Method* into the text box of the **Control**.


Press **F2** in the text box to open the *Method*. Then type in the source code of the **Control**.

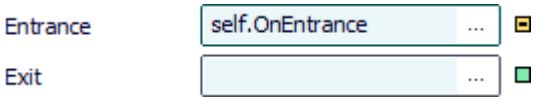
Instead of choosing **Select Object**, you can also select the *Method* in a *Frame*, drag it to the text box and drop it there.

#### Create a Control That is a Method of the Object

Proceed as follows to create a control as a *user-defined attribute* of data type *Method*:

- Type a meaningful name into the text box and select **Create Control [context menu]**. *Plant Simulation* then inserts `self.Name_you_typed_in_for_the_control`, such as `self.A1Ctrl`.


- Select **Create Control** on the empty text box. *Plant Simulation* then inserts `self.OnBuilt_in_name_of_the_control`, such as `self.OnEntrance`.




Type the source code of this control into the *Method* that opens.

To edit the source code later on:

  - Press **F2**.
  - Or hold down **Shift** and double-click into the text box.
  - Or select **Open Object** on the context menu.
  - Or click the tab **User-defined** and double-click the name of the *Method* in the list.
- To delete this control, delete the *user-defined attribute*. If you only delete the name from the text box, the *user-defined attribute* is retained.

### Remarks

The **Importer Request Control** and the **Exporter Request Control** are called in the following order:

- If you defined **Exporter and Importer Request Controls**, the **Exporter Request Controls** will be executed first. This way you can make the *Worker* follow the *part* when the *part* moves to the next station. This causes changed behavior as compared to modeling without controls. Without controls, the *Worker* is brokered for the new station in the *WorkerPool*.
- If several **Importer Request Controls** are executed at the same time, for example because an *Exporter*/*Worker* is released and if several open *Importers* exist, the sequence of the controls matches the sequence of the *Importers* in the list **Open Importers**. This way brokering of *Exporters*/*Workers* behaves identical when you model with our without controls.

### See also

**Importer Request Control [Broker]**

**Exporter Request Control [Broker]**

## Importer Request Control [Broker]

Modifies the built-in behavior of the object. The **Importer Request Control** sets how the *Broker* requests *importers*. The object calls the **Importer Request Control** whenever the *Broker* receives a request or when it would handle the request again, for example, when an *Exporter/Worker* registers as being available


### Remarks

*Plant Simulation* executes the **Importer Request Control** whenever the *Broker* receives a request or when it would handle the request again, for example, when an *Exporter/Worker* registers as being available. In the source code of the *Method* you yourself have to make sure that the request is taken care of, for example with the method **engage**. With this *control* you can model your own assignment strategies.

If you defined **Exporter and importer request controls**, the **Exporter Request Controls** will be executed first. This way you can make the *Worker* follow the *part* when the *part* moves to the next station. This causes changed behavior as compared to modeling without controls. Without controls, the *Worker* is brokered for the new station in the *WorkerPool*.

If several **Importer Request Controls** are executed at the same time, for example because an *Exporter/Worker* is released and if several open *Importers* exist, the sequence of the controls matches the sequence of the *Importers* in the list **Open Importers**. This way brokering of *Exporters/Workers* behaves identical when you model with our without controls.

The **Importer Request Control** does not interrupt methods that are executed at the moment. It will only be called after the current call chain has been executed. Importer requests can not overtake each other.

Compare this example:

- The **Processing Importer** of *Station2* cannot be satisfied because the *Worker* is already working at *Station1*.
- *Station1* finishes process the *part*. The *Worker* is released and the **Exit Control** of *Station1* is called. The **Importer Request Control** of *Station2* is only called after the **exit control** has been executed.
- The **Exit Control** of *Station1* moves the *part* to *Station3*. The **Importer Request Control** does not interrupt the **exit control**. After the **exit control** has been executed, the **Importer Request Control** of *Station2* is called and then the **Importer Request Control** of *Station3*.

**Note**

For *Broker* hierarchies *Plant Simulation* does not call the controls of the *sub-Brokers*, i.e., of several *Brokers* that are connected with *Connectors* in addition. *Plant Simulation* only calls the control of the *Broker* which you typed into the object.

### Select the Path to an Existing Method

Click the ellipsis button. Navigate to the location of the *Method* in the dialog **Select Object [for controls]** and click **OK**. This inserts the name of the *Method* into the text box of the **Control**.


Press **F2** in the text box to open the *Method*. Then type in the source code of the **Control**.

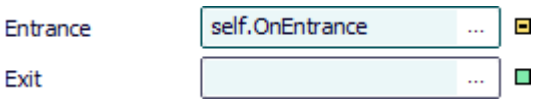
Instead of choosing **Select Object**, you can also select the *Method* in a *Frame*, drag it to the text box and drop it there.

### Create a Control That is a Method of the Object

Proceed as follows to create a control as a *user-defined attribute* of data type *Method*:

- Type a meaningful name into the text box and select **Create Control [context menu]**. *Plant Simulation* then inserts `self.Name_you_typed_in_for_the_control`, such as `self.A1Ctrl`.


- Select **Create Control** on the empty text box. *Plant Simulation* then inserts `self.OnBuilt_in_name_of_the_control`, such as `self.OnEntrance`.



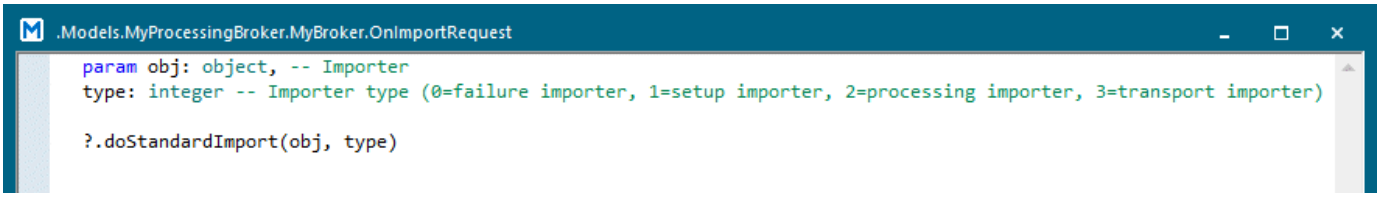

  Type the source code of this control into the *Method* that opens.

  To edit the source code later on:

  - Press **F2**.
  - Or hold down **Shift** and double-click into the text box.
  - Or select **Open Object** on the context menu.
  - Or click the tab **User-defined** and double-click the name of the *Method* in the list.

- To delete this control, delete the *user-defined attribute*. If you only delete the name from the text box, the *user-defined attribute* is retained.

The **standard importer request control** as a *user-defined attribute* looks like this:




**SimTalk**

**ImpRequestCtrl [SimTalk]**

**doStandardExport [SimTalk]**

**See also**

**Select Object [for controls]**

## Exporter Request Control [Broker]

Modifies the built-in behavior of the object. The *Method* sets how the *Broker* requests *Exporters/Workers*. The object calls the **Exporter Request Control** whenever an *Exporter/Worker* registers as being available with its *Broker*. It may then be assigned a new *importer*.


### Remarks

In the source code of the *Method* you yourself have to make sure that the request is taken care of, for example with the method **engage**. With this control you can assign a specific *importer* to the *Exporter/Worker*.

If you defined **Exporter and Importer request controls**, the **Exporter Request Controls** will be executed first. This way you can make the *Worker* follow the *part* when the *part* moves to the next station. This causes changed behavior as compared to modeling without controls. Without controls, the *Worker* is brokered for the new station in the *WorkerPool*.

If several Importer request controls are executed at the same time, for example because an *Exporter/Worker* is released and if several open *Importers* exist, the sequence of the controls matches the sequence of the *Importers* in the list **Open Importers**. This way brokering of *Exporters/Workers* behaves identical when you model with our without controls.

### Select the Path to an Existing Method

Click the ellipsis button. Navigate to the location of the *Method* in the dialog **Select Object [for controls]** and click **OK**. This inserts the name of the *Method* into the text box of the **Control**.


Press **F2** in the text box to open the *Method*. Then type in the source code of the **Control**.

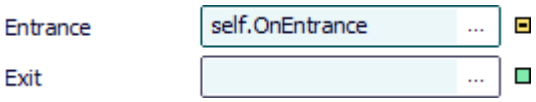
Instead of choosing **Select Object**, you can also select the *Method* in a *Frame*, drag it to the text box and drop it there.

### Create a Control That is a Method of the Object

Proceed as follows to create a control as a *user-defined attribute* of data type *Method*:

- Type a meaningful name into the text box and select **Create Control [context menu]**. *Plant Simulation* then inserts `self.Name_you_typed_in_for_the_control`, such as `self.A1Ctrl`.


- Select **Create Control** on the empty text box. *Plant Simulation* then inserts `self.OnBuilt_in_name_of_the_control`, such as `self.OnEntrance`.



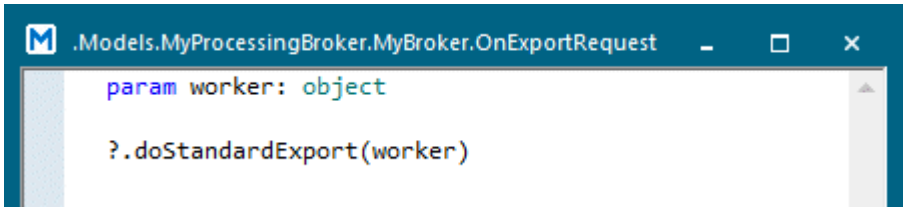

  Type the source code of this control into the *Method* that opens.

  To edit the source code later on:

  - Press **F2**.
  - Or hold down **Shift** and double-click into the text box.
  - Or select **Open Object** on the context menu.
  - Or click the tab **User-defined** and double-click the name of the *Method* in the list.

- To delete this control, delete the *user-defined attribute*. If you only delete the name from the text box, the *user-defined attribute* is retained.

The **standard exporter request control** as a *user-defined attribute* looks like this:




**SimTalk**

**ExpRequestCtrl [SimTalk]**

**doStandardExport [SimTalk]**

**See also**

**Select Object [for controls]**

## Tab Statistics

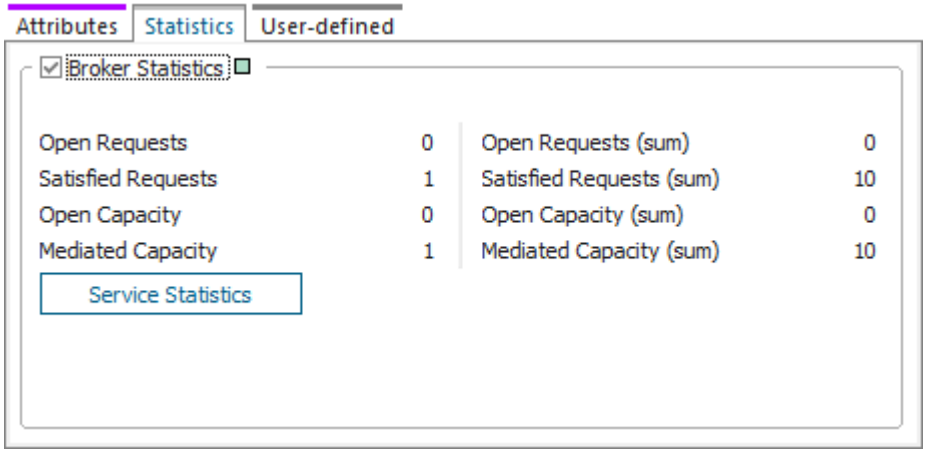
### Tab Statistics [Broker]

The tab **Statistics** shows the most important statistical data.

#### Remarks

To collect statistics data, select the check box **Broker Statistics**.

| Item English | Description | Read-only attribute | Item German |
|---|---|---|---|
| **Open Requests** | Shows the actual number of open requests. | OpenRequests | **Offene Anfragen** |
| **Open Requests (sum)** | Shows the total number of open requests. | StatOpenRequests | **Summe offener Anfragen** |
| **Satisfied Requests** | Shows the number of satisfied requests. | SatisfiedRequests | **Befriedigte Anfragen** |
| **Satisfied Requests (sum)** | Shows the total number of satisfied requests. | StatSatisfiedRequests | **Summe befriedigter Anfragen** |
| **Open Capacity** | Shows the amount of available capacities. | OpenCapacity | **Offene Kapazität** |
| **Open Capacity (sum)** | Shows the total amount of open capacities. | StatOpenCapacity | **Summe offener Kapazität** |
| **Mediated Capacity** | Shows the amount of brokered capacities. | MediatedCapacity | **Belegte Kapazität** |
| **Mediated Capacity (sum)** | Shows the total amount of brokered capacities. | StatMediatedCapacity | **Summe belegte Kapazität** |

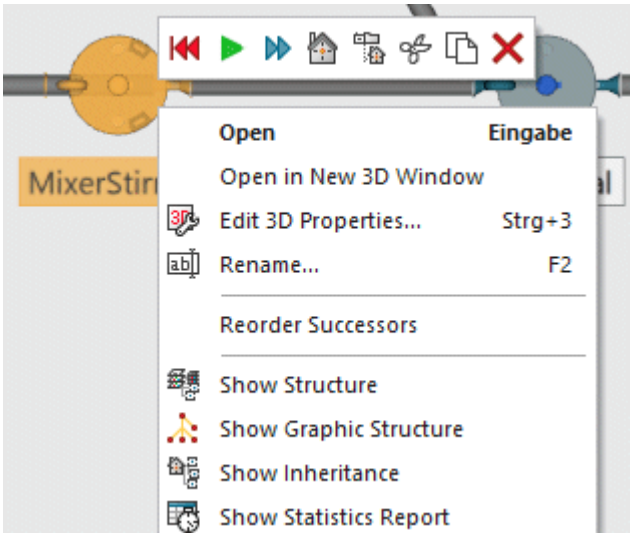


**Note**

For *Broker hierarchies*, i.e., for several *Brokers* connected with *Connectors*, the brokered services are only considered for the *Broker* which you typed into the *Importer*. This also applies if the service is provided by *sub-Brokers*.

To view **Broker Statistics** in the **Statistics Report**, select **View > Show Statistics Report** in the dialog of the object. The *Statistics Report* also shows the **Dwelling Time** and the **Mediation Time** per service of the *Broker*.

You can also click the right mouse button in the *Frame* and select **Show Statistics Report** or you can press **F6**.




**See also**

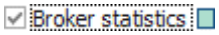
**brokerStat [SimTalk]**

**BrokerStatOn [SimTalk]**

**serviceStat [SimTalk]**

## Broker Statistics [activate]

To collect statistics data of the *Broker*, select this check box. To deactivate statistics collection, clear the checkbox.




### SimTalk

**BrokerStatOn [SimTalk]**

### See also

**Show Statistics Report [on View menu]**

**Time to Get Single Item**

## Service Statistics [Broker]

To open the service statistics table, which shows the **Dwelling Time** and the **Mediation Time** per service, click this button.


### Remarks

**Note**

If a service is fulfilled by several *Workers* or *Exporters/Workers*, *Plant Simulation* records this time interval only once. The same applies for the **count**. For the time during which the *Worker* was en-route to his job, statistics counts the time of the *Worker* who was en-route the longest.

| Item English | Description | Read-only attribute | Item German |
|---|---|---|---|
| **Services** | Shows the services, which the *Broker* brokered. | — | **Dienste** |
| **Dwelling Time: Count** | Shows how often the service in this row was brokered, i.e., how often the *Exporter/Worker* stayed at the *importer/importers*. | — | **Verweildauer: Anzahl** |
| **Sum** | Shows the sum of the times during which the service in this row was brokered. | StatStayTime | **Summe** |
| **Mean Value** | Shows the mean duration of the times during which the service in this row was brokered. | StatStayTimeMu | **Mittelwert** |
| **Standard Deviation** | Shows the standard deviation from the mean value of the times during which the service in this row was brokered. | StatStayTimeDelta | **Standardabweichung** |
| **Min** | Shows the minimum time during which the service in this row was brokered. | — | **Min** |
| **Max** | Shows the maximum time during which the service in this row was brokered. | — | **Max** |
| **Mediation Time: Count** | Shows how often the *importer/importers* were waiting for the procurement of the service in this row. | — | **Vermittlungsdauer: Anzahl** |
| **Sum** | Shows the sum of the times during which the *importer/importers* were waiting for the procurement of the service in this row. | StatMediationTime | **Summe** |
| **Mean Value** | Shows the mean duration of the times during which the *importer/importers* were waiting for the procurement of the service in this row. | StatMediationTimeMu | **Mittelwert** |

| Item English | Description | Read-only attribute | Item German |
|---|---|---|---|
| **Standard Deviation** | Shows the standard deviation from the mean value of the times during which the *importer/importers* were waiting for the procurement of the service in this row. | StatMediationTimeDelta | **Standardabweichung** |
| **Min** | Shows the minimum time during which the *importer/importers* were waiting for the procurement of the service in this row. | — | **Min** |
| **Max** | Shows the maximum time during which the *importer/importers* were waiting for the procurement of the service in this row. | — | **Max** |

**SimTalk**

**serviceStat [SimTalk]**

**See also**

**Broker Statistics [described]**

*Statistics report*, **Broker Statistics [described]**

## Tab User-defined

Define your own attributes as described under the **Tab User-defined**.

### Navigate Menu

The commands are described under the **Navigate Menu**.

### View Menu

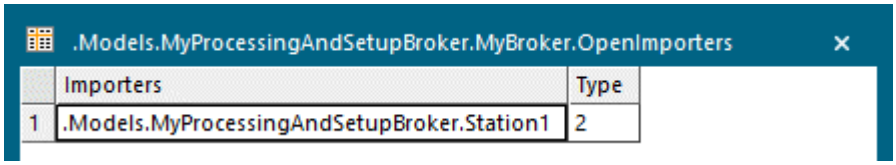
### View Menu [Broker]

The **View Menu** provides commands to access its functions.

| | |
|---|---|
| **Refresh [on View menu]** | **Satisfied Importers [Broker]** |
| **Show Statistics Report [on View menu]** | **Exporters [Broker]** |
| **Show Attributes and Methods [on View menu]** | **Offered Services [Broker]** |
| **Open Importers [Broker]** | |

## Open Importers [Broker]

Opens a table, which shows all objects whose *importers* registered an unsatisfied request with the *Broker*.




### Remarks

The table shows the **Importers** in column 1, and their **Type** in column 2: The number 0 designates the *failure-importer*, 1 the *set-up-importer*, 2 the *processing-importer*, and 3 the *transport-importer*.

An *Importer*, who is interrupted because another station with a higher priority requires the resource, is sorted at the first place in the list.

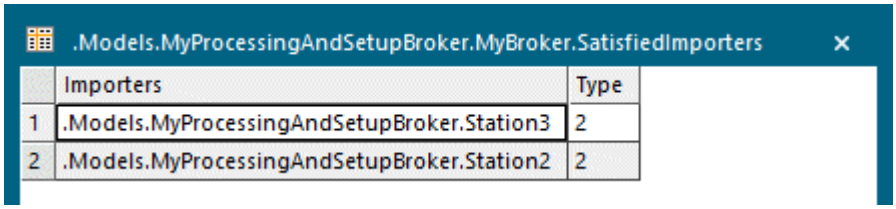
### See also

**getOpenImporters [SimTalk]**

**forgetOpenRequest [SimTalk]**

## Satisfied Importers [Broker]

Opens a table, which shows all objects whose *importers* were assigned *Exporters/Workers*.




### Remarks

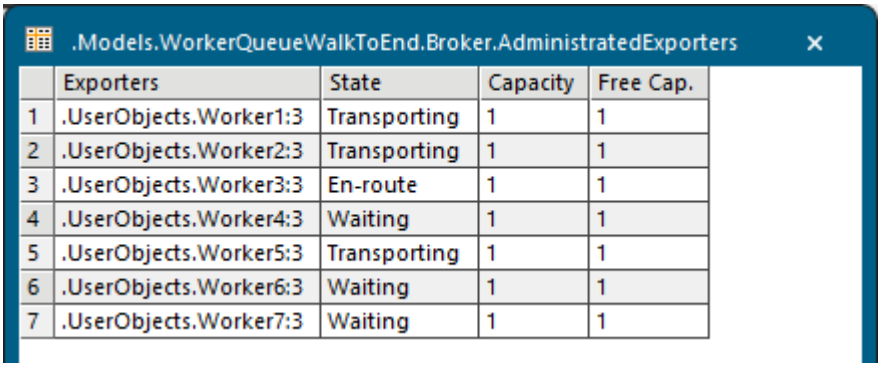
The table shows the **Importers** in column 1, and their **Type** in column 2: The number 0 designates the *failure-importer*, 1 the *set-up-importer*, 2 the *processing-importer*, and 3 the *transport-importer*.

### See also

**getSatisfiedImporters [SimTalk]**

## Exporters [Broker]

Opens a table, which shows information about all *Exporters* registered with the *Broker*.




| | Exporters | State | Capacity | Free Cap. |
|---|---|---|---|---|
| 1 | .UserObjects.Worker1:3 | Transporting | 1 | 1 |
| 2 | .UserObjects.Worker2:3 | Transporting | 1 | 1 |
| 3 | .UserObjects.Worker3:3 | En-route | 1 | 1 |
| 4 | .UserObjects.Worker4:3 | Waiting | 1 | 1 |
| 5 | .UserObjects.Worker5:3 | Transporting | 1 | 1 |
| 6 | .UserObjects.Worker6:3 | Waiting | 1 | 1 |
| 7 | .UserObjects.Worker7:3 | Waiting | 1 | 1 |

### Remarks

The column **Exporters** shows the names of the *Exporters/Workers*.

The column **State** shows the state of the respective *Exporter/Worker*.

The column **Capacity** shows the capacity of the respective *Exporter/Worker*.

The column **Free Cap.** shows the free capacity of the respective *Exporter/Worker*.

### See also

**AdministeredExporters [SimTalk]**

## Offered Services [Broker]

Opens a table, which shows information about the services offered by the *Exporters*/*Workers* which are managed by the *Broker*.

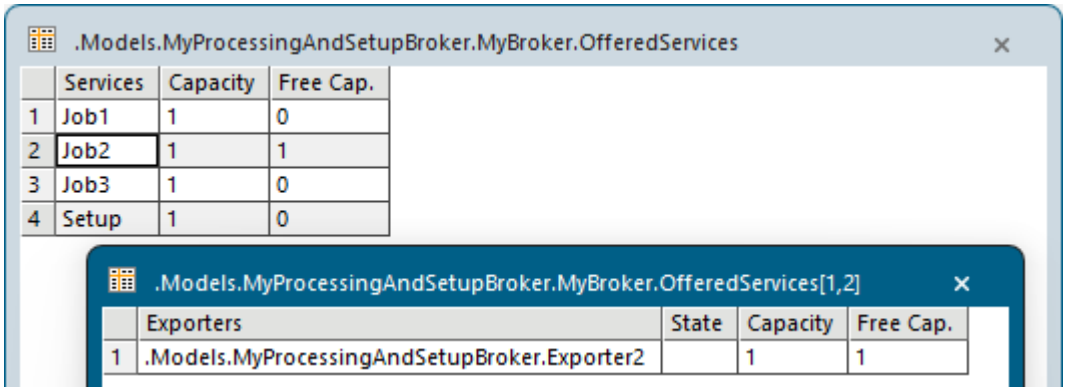
### Services Offered by the Exporter

The column **Services** shows the name of the service which the *Exporter* offers.

The column **Capacity** shows the overall capacity which the *Exporter* offers.

The column **Free Cap.** shows the free capacity which the *Exporter* offers.

Double-click the name of a service to open a subtable, which shows the path of the *Exporter*.




The column **State** shows the state of the respective *Exporter*.

The column **Capacity** shows the capacity of the respective *Exporter*.

The column **Free Cap.** shows the free capacity of the respective *Exporter*.

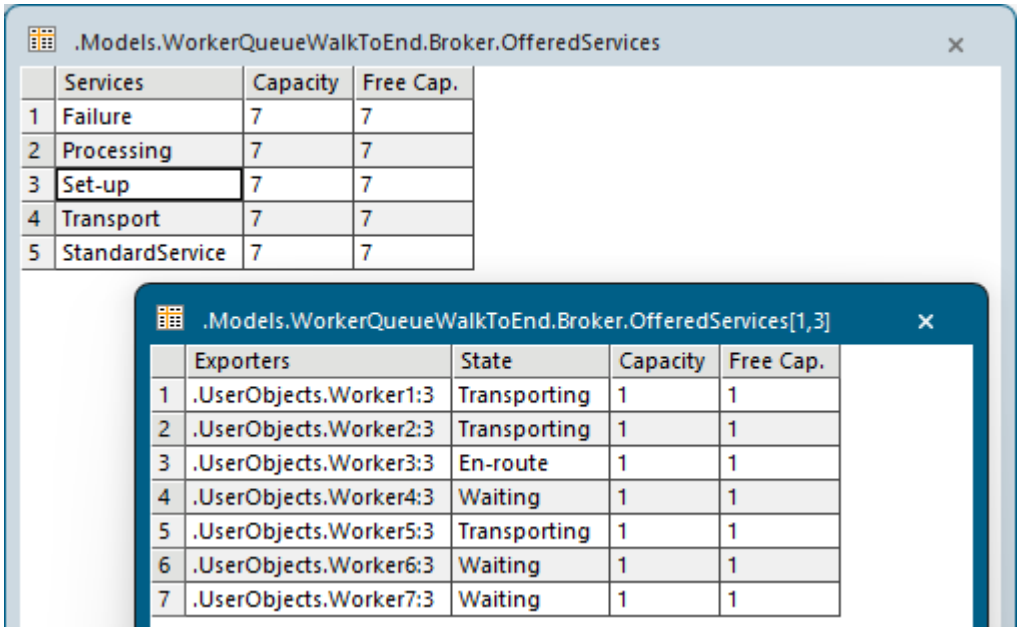
### Services Offered by the Worker

The column **Services** shows the name of the service which the *Worker* offers.

The column **Capacity** shows the overall capacity which the *Worker* offers.

The column **Free Cap.** shows the free capacity which the *Worker* offers.

Double-click the name of a service to open a subtable, which shows the path of the individual *Worker*.




The column **State** shows the state of the respective *Worker*.

The column **Capacity** shows the capacity of the respective *Worker*.

The column **Free Cap.** shows the free capacity of the respective *Worker*.

**See also**

**getOfferedServices [SimTalk]**

## Tools Menu

The **Tools Menu** provides these menu commands:

**Edit Controls**

**Edit Observers**

### Help Menu

The commands are described under the **Help Menu**.

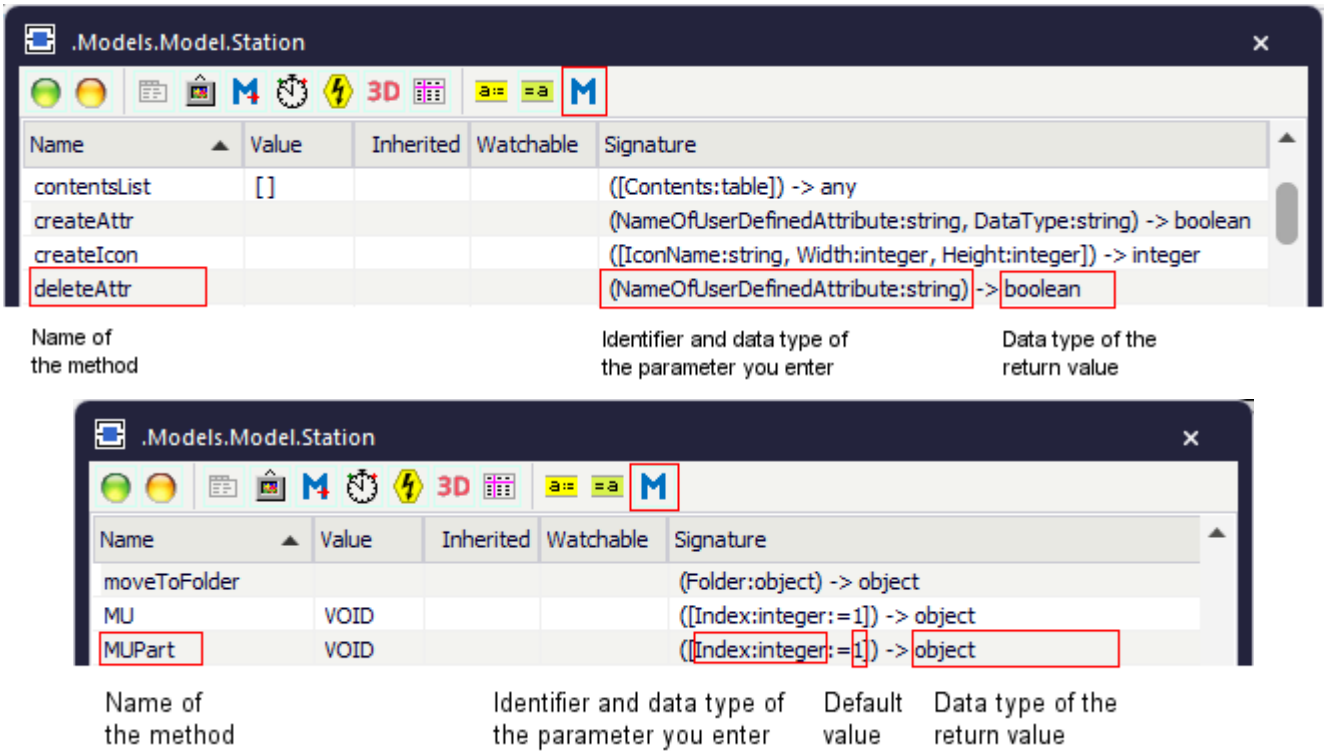
## Methods of the Broker

### _Methods of the Broker

The *Broker* provides:

- The *methods* listed in the table of contents to the left.
- The **Methods of All Objects**.

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window **Show Attributes and Methods**. The figure below illustrates the information using the example of the object *Station*.




- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *methods*, *read-only attributes*, and *attributes* of the selected **Class [general description]**.
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *methods*, *read-only attributes*, and *attributes* of the selected **Instance [general description]**.